

文章笔记

Harness Engineering: Agent-First 开发范式下的工程实践

基于 OpenAI 技术博客整理

2026 年 4 月 2 日

“Humans steer. Agents execute.”

— Ryan Lopopolo, OpenAI

文章作者: Ryan Lopopolo (OpenAI)

发布日期: 2026-02-11

预计阅读时长: 约 20 分钟

文章链接: <https://openai.com/index/harness-engineering/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：一个激进的实验	2
1.1 实验数据概览	2
1.2 本章小结	2
2 工程师角色的重新定义	4
2.1 从写代码到设计环境	4
2.2 深度优先的构建策略	4
2.3 人类与 Agent 的交互模式	4
2.4 本章小结	5
3 上下文管理：给 Agent 一张地图	6
3.1 “一个大文件”方法的失败	6
3.2 地图而非百科全书	6
3.3 渐进式披露 (Progressive Disclosure)	7
3.4 机械化的知识库维护	7
3.5 本章小结	7
4 Agent 可读性：让代码库对 Agent “可见”	8
4.1 以 Agent 可读性为第一优先	8
4.2 提升应用可读性的具体措施	8
4.3 技术选型偏好	9
4.4 本章小结	9
5 架构执行与品味守护	10
5.1 机械化的架构约束	10
5.2 自定义 Linter 与品味不变量	10
5.3 约束与自由的平衡	10
5.4 本章小结	11
6 吞吐量革命：合并哲学与熵管理	12
6.1 当吞吐量改变合并哲学	12
6.2 Agent 产出的全面性	12
6.3 不断增长的自主性	12
6.4 熵与“垃圾回收”	13
6.5 本章小结	13
7 总结与延伸	14
7.1 核心经验汇总	14
7.2 开放问题	14
7.3 对从业者的启示	14
7.4 延伸阅读	14

1 引言：一个激进的实验

2025 年 8 月，OpenAI 内部一个小团队启动了一项激进的实验：从一个空的 Git 仓库开始，构建一个完整的软件产品，且**不写一行人工代码**。五个月后，这个仓库包含了约一百万行代码，涵盖应用逻辑、基础设施、工具链、文档以及内部开发者工具。在此期间，一个仅三人的工程团队通过 Codex 提交并合并了约 1,500 个 Pull Request，平均每人每天 3.5 个 PR。该产品已在 OpenAI 内部拥有数百名用户，包括每日活跃的重度用户。

核心理念：Harness Engineering

Harness Engineering（驾驭工程）是文章提出的核心概念。当 AI Agent 成为主要的代码产出者时，软件工程师的角色从“写代码”转变为“设计环境、表达意图、构建反馈循环”。人类驾驭（harness）Agent，而非亲自执行编码任务。这个名字来源于马具（harness）的隐喻：你不替马跑，而是为它装上缰绳和方向盘。

这篇文章是 OpenAI 内部团队的一线实战报告，记录了在完全 Agent 驱动的开发模式下遇到的问题、积累的经验、以及逐步形成的方法论。它回答了一个关键问题：**当工程师不再写代码时，他们应该做什么？**

背景：Codex 与 Agent-First 开发

Codex 是 OpenAI 推出的 coding agent 产品，能够理解自然语言指令并生成代码、运行测试、提交 PR。“Agent-First”开发范式指的是将 AI Agent 作为主要的代码生产者，人类工程师转而负责系统设计、质量保障和方向把控。这与传统的“AI 辅助编程”（如 Copilot 自动补全）有本质区别：后者是人写代码、AI 辅助，前者是 AI 写代码、人类引导。

1.1 实验数据概览

指标	数据
项目起始时间	2025 年 8 月底
代码规模	约 100 万行
PR 总数	约 1,500 个
初始团队规模	3 名工程师
后续团队规模	7 名工程师
人均日 PR 数	3.5 个
人工代码行数	0 行
估算提速倍数	约 10 倍
单次 Codex 最长运行时间	6 小时以上

表 1: OpenAI Harness Engineering 实验关键数据

1.2 本章小结

OpenAI 团队通过一个极端约束（零人工代码）验证了 Agent-First 开发范式的可行性。关键发现是：Agent 产出代码的速度远超人类，但**真正的瓶颈不在于代码生成，而在于人类的注意力和判**

断力。整篇文章围绕如何最大化利用这一稀缺资源展开。

2 工程师角色的重新定义

2.1 从写代码到设计环境

传统软件工程中，工程师的核心产出是代码。在 Agent-First 模式下，这一角色发生了根本性转变。

工程师的新职责

在 Agent-First 开发中，工程师的主要工作变为：

- **设计环境 (Environment Design)**：构建 Agent 能有效工作的基础设施
- **表达意图 (Intent Specification)**：将需求转化为 Agent 可理解的 Prompt
- **构建反馈循环 (Feedback Loops)**：让 Agent 能自我验证和迭代
- **识别能力缺口**：当 Agent 失败时，分析缺少什么工具或抽象

文章指出，早期进展比预期慢，**原因不是 Codex 能力不足，而是环境欠缺规范 (underspecified)**。Agent 缺乏完成高层目标所需的工具、抽象和内部结构。因此，工程团队的首要任务变成了“赋能 Agent 做有用的工作”。

2.2 深度优先的构建策略

实践中，团队采用**深度优先**的构建策略：

1. 将大目标分解为更小的构建块（设计、编码、评审、测试等）
2. 提示 Agent 构建这些基础模块
3. 用这些模块解锁更复杂的任务

当某个步骤失败时，修复方式几乎从来不是“再试一次”。因为唯一的推进方式是让 Codex 完成工作，人类工程师总是退一步问自己：“**缺少什么能力？如何让它对 Agent 既可见 (legible) 又可执行 (enforceable)？**”

常见误区：把 Agent 当作更快的人类开发者

很多团队在尝试 Agent 开发时，仍然用人类开发者的思维模式来使用 Agent。例如直接给一个模糊的大需求，期望 Agent 能像资深工程师一样自主分解和执行。文章的经验表明，这种方式会失败。Agent 需要的是**结构化的环境和明确的约束**，而不是更好的指令。“Try harder”几乎从来不是正确的解法。

2.3 人类与 Agent 的交互模式

工程师几乎完全通过 Prompt 与系统交互。一个典型的工作流程如下：

1. 工程师描述一个任务
2. 运行 Agent，让它开启一个 Pull Request
3. 指示 Codex 在本地自我审查变更
4. 请求其他 Agent 进行额外评审（本地和云端）
5. 响应人类或 Agent 的反馈
6. 在循环中迭代，直到所有 Agent 评审者满意
7. 合并变更

值得注意的是，人类**可以但不必须**审查 Pull Request。随着时间推移，团队将几乎所有审查工作推向了 Agent-to-Agent 的模式。

Ralph Wiggum Loop

文章中提到了一个有趣的术语“Ralph Wiggum Loop”，指的是 Agent 自我审查、收到反馈、修改、再审查的循环迭代过程。这个名称来自《辛普森一家》中的角色 Ralph Wiggum，暗含一种自我对话的幽默感。这种循环是 Agent-First 开发中质量保障的核心机制。

2.4 本章小结

在 Agent-First 范式下，工程师的角色从“代码编写者”变为“系统架构师 + Agent 教练”。核心技能从编程转变为环境设计、意图表达和反馈循环构建。关键原则是：当 Agent 失败时，不要“试更多次”，而是退一步问“缺少什么基础设施”。

3 上下文管理：给 Agent 一张地图

上下文管理是让 Agent 高效处理大型复杂任务的最大挑战之一。OpenAI 团队在这方面积累了深刻的教训。

3.1 “一个大文件”方法的失败

团队最初尝试将所有指导信息放入一个大型 AGENTS.md 文件中。这种方法以可预见的方式失败了：

巨型 AGENTS.md 的四大问题

- 1. **上下文是稀缺资源**：一个巨型指令文件会挤占任务本身、代码和相关文档的空间，导致 Agent 遗漏关键约束或优化错误的目标。
- 2. **过多指导等于没有指导**：当所有东西都是“重要”的，就没有东西是重要的。Agent 最终只会进行局部模式匹配，而非有意识地导航。
- 3. **瞬间腐烂**：一个单体手册很快变成过时规则的坟场。Agent 无法判断哪些仍然有效，人类也懒得维护，文件悄然变成一个“有吸引力的危险物”（attractive nuisance）。
- 4. **难以验证**：单一大文件不利于机械化检查（覆盖率、新鲜度、归属、交叉链接），因此漂移不可避免。

3.2 地图而非百科全书

团队转而采用“目录 + 深层知识库”的架构：

AGENTS.md 作为目录（Table of Contents）

将 AGENTS.md 视为**目录**而非百科全书。它应当是简短的（约 100 行），仅包含指向更深层信息源的指针。仓库的知识库存放在结构化的 docs/ 目录中，作为“唯一真实来源”（system of record）。

文章给出的仓库知识架构如下：

文件/目录	用途
AGENTS.md	约 100 行的入口地图，指向深层文档
ARCHITECTURE.md	顶层架构图，描述域与包的层次关系
docs/design-docs/	设计文档索引，含验证状态和核心信念
docs/exec-plans/	执行计划（活跃/已完成），含决策日志
docs/generated/	自动生成的文档（如数据库 schema）
docs/product-specs/	产品规格说明书
docs/references/	外部参考资料的 LLM 友好版本
QUALITY_SCORE.md	各产品域和架构层的质量评分
RELIABILITY.md	可靠性要求
SECURITY.md	安全规范

表 2: 知识库目录结构及用途

3.3 渐进式披露 (Progressive Disclosure)

这种架构实现了**渐进式披露**：Agent 从一个小而稳定的入口点开始，被教导在需要时去哪里查找更多信息，而不是一开始就被信息淹没。

渐进式披露的设计理念

渐进式披露 (Progressive Disclosure) 是交互设计中的经典原则，指的是只在用户需要时才展示详细信息。在 Agent 上下文管理中，这意味着：Agent 的初始上下文只包含“地图”级别的信息，具体细节由 Agent 在任务执行过程中按需获取。这与人类新员工入职的逻辑相同——先给组织架构图，再按需深入各个模块。

3.4 机械化的知识库维护

团队不仅依赖人工维护知识库，还通过**机械化手段**确保其质量：

- 专用的 Linter 和 CI Job 验证知识库是否最新、交叉链接正确、结构合规
- 定期运行的“文档园丁” (doc-gardening) Agent 扫描过时或不反映真实代码行为的文档
- 发现问题后自动开启修复性 Pull Request

3.5 本章小结

上下文管理的核心原则是“给 Agent 一张地图,而非一本百科全书”。具体实践包括:将 AGENTS.md 保持简短作为入口指针;将详细知识存入结构化的 docs/ 目录;采用渐进式披露策略;并通过 Linter、CI 和自动化 Agent 机械化地维护知识库的新鲜度和一致性。

4 Agent 可读性：让代码库对 Agent “可见”

4.1 以 Agent 可读性为第一优先

因为仓库完全由 Agent 生成，它首先为 **Codex 的可读性** 做优化——而不是人类的阅读习惯。这类似于团队为新入职工程师优化代码可导航性，只是对象换成了 Agent。

Agent 可读性的核心原则

从 Agent 的视角看，任何它在运行时无法在上下文中访问的东西，实际上都不存在。存放在 Google Docs 里的讨论、Slack 聊天记录、或人们脑中的决策——这些对 Agent 都是不可见的。只有仓库本地的、版本化的产出物（代码、Markdown、Schema、可执行计划）才是 Agent 能看到的。

这意味着团队需要不断将更多上下文**推入仓库**中。那次在 Slack 上对齐团队架构方向的讨论？如果 Agent 发现不了，它就像一个三个月后加入的新人——对此一无所知。

4.2 提升应用可读性的具体措施

随着代码产出量的增加，瓶颈转移到了人类的 QA 能力上。团队通过以下方式让更多能力直接对 Agent 可见：

措施	效果
每个 Git worktree 可独立启动应用	Codex 可以为每个变更启动和驱动一个独立实例
接入 Chrome DevTools Protocol	Agent 可以操作 DOM 快照、截图和页面导航
本地可观测性栈	每个 worktree 有独立的日志、指标和追踪，任务完成后销毁
LogQL / PromQL 查询能力	Agent 可以直接查询日志和指标

表 3: 提升 Agent 可读性的关键措施

这些能力使得以下类型的 Prompt 成为可能：

- “确保服务启动在 800ms 内完成”
- “这四个关键用户旅程中没有任何 span 超过两秒”

Agent 可以直接复现 Bug、验证修复、并对 UI 行为进行推理——这些曾经需要人类手动完成的 QA 工作现在被自动化了。

4.3 技术选型偏好

“无聊技术”对 Agent 更友好

团队发现，通常被描述为“无聊的”（boring）技术往往更容易被 Agent 建模，原因包括：

- **可组合性**（composability）：简单接口易于推理
- **API 稳定性**：不会因为版本变化而打破 Agent 的既有认知
- **训练集覆盖率**：经典技术在 LLM 训练数据中有更充分的表示

在某些情况下，让 Agent 重新实现功能子集比使用不透明的第三方库更划算。例如，团队没有使用通用的 `p-limit` 式并发控制包，而是自行实现了一个并发映射辅助函数，它与 Open-Telemetry 监控紧密集成、100% 测试覆盖、且行为完全符合运行时预期。

4.4 本章小结

Agent 可读性是 Agent-First 开发的基础设施层。核心原则是：Agent 在运行时看不到的东西等于不存在。团队需要持续将讨论、决策、知识推入版本化的仓库产出物中，并通过工具链（DevTools、可观测性栈等）让 Agent 能直接与应用交互。技术选型上，优先选择“无聊但稳定”的技术。

5 架构执行与品味守护

5.1 机械化的架构约束

文档本身无法让一个完全由 Agent 生成的代码库保持一致性。团队通过**执行不变量 (invariants)**，而非微管理实现的方式来解决这一问题。

严格的分层架构模型

每个业务域被划分为固定的层次集合，层间依赖方向严格验证，只有有限的合法边集。具体分层为：

Types → Config → Repo → Service → Runtime → UI

跨切面关注点 (auth、connectors、telemetry、feature flags) 通过单一显式接口 **Providers** 注入。其他任何依赖关系都被禁止，并通过机械化手段强制执行。

文章特别指出：**这种严格的架构通常是拥有数百名工程师的大型团队才会考虑的事情**。但在 Agent 开发中，它是早期前提——正是这些约束使得高速开发不会导致衰退或架构漂移。

5.2 自定义 Linter 与品味不变量

团队使用自定义 Linter（同样由 Codex 生成）和结构化测试来执行规则：

强制规则类型	示例
结构化日志	静态强制所有日志输出遵循结构化格式
命名约定	Schema 和类型的命名必须遵循统一规范
文件大小限制	防止单文件过大，保持可读性
平台特定可靠性要求	针对不同平台的性能和可靠性基线
数据边界验证	要求在数据边界解析数据形状（如使用 Zod）
依赖方向验证	代码只能“向前”依赖固定的层次顺序

表 4: 品味不变量 (Taste Invariants) 示例

自定义 Linter 的关键设计：错误消息即 Agent 指令

因为 Linter 是自定义的，团队将错误消息设计为包含**修复指导** (remediation instructions)，这些指导会直接注入 Agent 的上下文中。这意味着 Lint 错误不仅告诉 Agent “你做错了什么”，还告诉它“应该怎么修”。在人类开发中，这些规则可能显得教条；在 Agent 开发中，它们是乘数效应——一旦编码，即可在所有地方同时生效。

5.3 约束与自由的平衡

团队的理念类似于领导一个大型工程平台组织：

- **中央强制执行**：边界、正确性、可复现性
- **局部允许自治**：在约束范围内，Agent 有充分自由决定解决方案的表达式

结果是代码并不总是符合人类的风格偏好——**但这没关系**。只要输出是正确的、可维护的、且对未来的 Agent 运行可读，就达到了标准。

人类的品味通过以下渠道持续反馈到系统中：

1. Review 评论
2. 重构 Pull Request
3. 面向用户的 Bug 被捕获为文档更新
4. 当文档不够时，将规则提升为代码（Linter）

5.4 本章小结

在 Agent-First 开发中，架构约束不是事后添加的技术债务治理工具，而是**速度的前提条件**。通过严格的分层架构、自定义 Linter（带修复指导的错误消息）、以及“中央强制边界、局部允许自治”的平衡策略，团队在保持高速产出的同时防止了架构衰退。

6 吞吐量革命：合并哲学与熵管理

6.1 当吞吐量改变合并哲学

随着 Codex 产出速度的提高，许多传统的工程规范变得**适得其反**。

高吞吐量环境下的合并哲学

仓库运行在最小化的阻塞合并门控（blocking merge gates）模式下：

- Pull Request 是短周期的
- 测试的 flake（不稳定失败）通常通过后续运行解决，而非无限期阻塞
- 在 Agent 吞吐量远超人类注意力的系统中，**修正是廉价的，等待是昂贵的**

警告：这在低吞吐量环境中是不负责任的

文章明确指出，这种“快速合并、后续修正”的策略**在低吞吐量环境中是不负责任的**。它只有在以下条件同时满足时才是合理的权衡：

- Agent 产出速度远超人类审查速度
- 修正的成本极低（Agent 可以快速修复）
- 架构约束提供了安全网（严格的 Linter 和测试）
- 每个变更是隔离的（独立的 worktree 和环境）

不满足这些前提时，照搬这种做法会导致灾难。

6.2 Agent 产出的全面性

当文章说“代码库由 Codex Agent 生成”时，指的是**代码库中的所有东西**：

产出类别	说明
产品代码和测试	应用逻辑与对应的自动化测试
CI 配置和发布工具	持续集成管线和部署工具链
内部开发者工具	辅助团队日常工作的脚本和工具
文档和设计历史	架构决策记录与设计文档
评估框架（Eval Harness）	用于评估 Agent 产出质量的框架
Review 评论和回复	PR 上的代码审查交互
仓库管理脚本	管理仓库本身的自动化脚本
生产环境 Dashboard 定义	监控面板的配置文件

表 5: Codex Agent 的全部产出类别

人类始终保持在循环中，但工作在不同的抽象层次：优先排序工作、将用户反馈转化为验收标准、以及验证结果。

6.3 不断增长的自主性

随着越来越多的开发循环被直接编码到系统中，仓库最近跨越了一个有意义的门槛：Codex 可以端到端地驱动一个新功能。给定单个 Prompt，Agent 现在可以：

1. 验证代码库的当前状态
2. 复现一个被报告的 Bug
3. 录制一段视频展示失败场景
4. 实现修复
5. 通过驱动应用验证修复
6. 录制第二段视频展示修复后的结果
7. 开启一个 Pull Request
8. 响应 Agent 和人类的反馈
9. 检测并修复构建失败
10. 仅在需要人类判断时才升级
11. 合并变更

注意范围限定

文章特别强调，这种高度自主的行为**严重依赖于该仓库的特定结构和工具链**，不应假设它能在没有类似投入的情况下泛化到其他项目——至少目前还不能。这是一个关于“什么是可能的”的演示，而非“所有人都应该这样做”的建议。

6.4 熵与“垃圾回收”

完全 Agent 自主也引入了新问题。Codex 会**复制仓库中已存在的模式**——包括不均匀或次优的模式。随着时间推移，这不可避免地导致漂移。

最初，团队手动处理这个问题：**每周五花 20% 的时间清理“AI 垃圾” (AI slop)**。不出所料，这种方式无法扩展。

Golden Principles + 自动化垃圾回收

团队转而采用“黄金原则 + 自动清理”的策略：

1. 将“黄金原则” (golden principles) 编码到仓库中——这些是保持代码库对未来 Agent 运行可读和一致的主观但机械化的规则
2. 定期运行后台 Codex 任务，扫描偏差、更新质量评分、开启针对性的重构 PR
3. 大多数这类 PR 可以在一分钟内审查并自动合并

技术债务就像高利贷：**以小额持续还款几乎总比让它复利累积后痛苦地一次性偿还要好。**

黄金原则的两个具体示例：

- **偏好共享工具包**：使用共享 utility package 而非手写 helper，以集中不变量
- **不做 YOLO 式数据探测**：在边界验证数据或依赖类型化 SDK，防止 Agent 基于猜测的数据形状构建逻辑

6.5 本章小结

高吞吐量环境下，“修正廉价、等待昂贵”成为合并哲学的核心。但这需要严格的架构约束作为安全网。Agent 会复制已有模式（包括坏模式），因此需要“垃圾回收”机制：将品味编码为黄金原则，通过自动化 Agent 持续扫描和修复偏差，实现技术债务的持续小额偿还。

7 总结与延伸

7.1 核心经验汇总

OpenAI 团队的 Harness Engineering 实验揭示了 Agent-First 开发范式的若干核心原则：

原则	内涵
人类驾驭，Agent 执行	工程师从写代码转向设计环境、表达意图、构建反馈循环
地图优于百科全书	AGENTS.md 应是简短的目录，知识库分布式存储
可见性决定存在性	Agent 在运行时看不到的东西等于不存在
约束是速度的前提	严格的架构约束使高速产出不致衰退
修正廉价，等待昂贵	高吞吐量下快速合并、后续修正优于阻塞等待
持续垃圾回收	技术债务小额持续偿还，不让其复利累积
无聊技术更可靠	稳定、简单、训练集覆盖充分的技术对 Agent 更友好

表 6: Harness Engineering 核心原则总结

7.2 开放问题

文章坦诚地指出了几个尚未回答的问题：

1. **长期架构一致性**：在完全 Agent 生成的系统中，架构的一致性如何随年数演进？五个月和五年是完全不同的时间尺度。
2. **人类判断的最优杠杆点**：人类判断在哪些环节提供最大的杠杆效应？如何编码这些判断使其能够持续复利？
3. **模型能力迭代的影响**：随着模型持续变得更强大，这套系统如何演进？当前的很多约束是否会变得不再必要？

7.3 对从业者的启示

从这篇文章中可以立即行动的三件事

1. **审视你的 AGENTS.md / CLAUDE.md**：它是地图还是百科全书？如果超过 200 行，考虑拆分为结构化的 docs/ 目录。
2. **将隐性知识推入仓库**：那些只存在于 Slack、会议纪要或人们脑中的架构决策，值得被文档化为版本控制的 Markdown。
3. **投资自定义 Linter**：特别是带有修复指导的错误消息，它们是“一次编码、处处生效”的乘数效应工具。

7.4 延伸阅读

- OpenAI Codex 官方文档：<https://openai.com/codex>
- Anthropic 关于 CLAUDE.md 最佳实践的讨论
- Martin Fowler 关于“Strangler Fig Pattern”的文章——渐进式重构与 Agent 垃圾回收有相似的哲学
- “Choose Boring Technology” (Dan McKinley, 2015)——文章中“无聊技术对 Agent 更友好”的论点的早期理论基础

“Building software still demands discipline, but the discipline shows up more in the scaffolding rather than the code.”

— Ryan Lopopolo, OpenAI